ClickSense

A ClickHouse MCP server with a semantic layer

User Guide — version 1.0.0

What this guide covers. Installing and running ClickSense, connecting it to an MCP client, the recommended agent workflow, the reference of every tool, resource, prompt and skill, how to author the semantic layer for your own warehouse, the safety model that keeps ClickHouse up, performance tuning, operations and troubleshooting.

Who it is for. Data and platform engineers who deploy the server, analytics engineers who curate the semantic layer, and anyone who builds agents on top of it.

Audience	Start with
Trying it out	Chapter 2 (Quick start), then Chapter 4 (Workflow)
Deploying to production	Chapters 3 (Configuration) and 9 (Safety model)
Curating business knowledge	Chapters 7 (Authoring) and 8 (Disambiguation)
Building agents on top	Chapters 4 (Workflow), 5 (Tools) and 6 (Resources)

Generated from docs/build_guide.py — rebuild with: .venv/bin/python docs/build_guide.py

Contents

1. Overview 4

1.1 Architecture . . . 4

1.2 Key capabilities . . . 4

2. Installation and quick start 6

2.1 Requirements . . . 6

2.2 Install the server . . . 6

2.3 Start the demo database . . . 6

2.4 Configure and check . . . 6

2.5 Connect an MCP client . . . 6

2.6 Command-line reference . . . 7

3. Configuration 8

3.1 connection . . . 8

3.2 limits . . . 8

3.3 policy . . . 9

3.4 Other keys . . . 9

4. The recommended agent workflow 10

4.1 Worked example . . . 10

5. Tool reference 11

5.1 Discovery . . . 11

5.2 Querying . . . 13

5.3 Control . . . 14

5.4 Disambiguation . . . 15

5.5 Operations and learning . . . 16

6. Resources, prompts and skills 17

6.1 Resources . . . 17

6.2 Prompts . . . 17

6.3 Skills shipped . . . 17

7. Authoring the semantic layer 18

7.1 Data dictionary . . . 18

7.2 Glossary . . . 19

7.3 Metrics . . . 19

7.4 Examples and knowledge . . . 19

7.5 Skills . . . 20

7.6 The inbox loop . . . 20

8. Disambiguation in depth 21

8.1 Kinds of ambiguity and how each is surfaced . . . 21

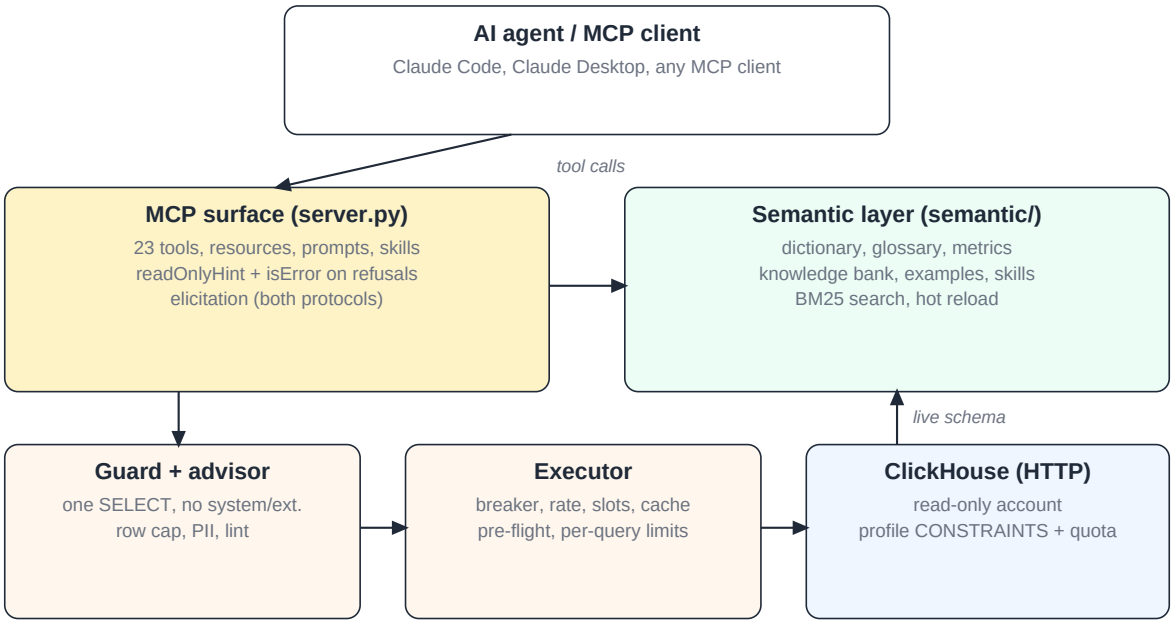
8.2 How values are resolved . . . 21
9. Safety model: not taking the database down 22
9.1 Line 1 — the ClickHouse account . . . 22
9.2 Line 2 — the server, before sending . . . 22
9.3 Line 3 — every query carries its limits . . . 23
9.4 PII . . . 23
10. Performance and tuning 24
10.1 Measured latencies . . . 24
10.2 What makes it fast . . . 24
10.3 Tuning guide . . . 24
11. Operations and troubleshooting 25
11.1 Audit log . . . 25
11.2 Error codes . . . 25
11.3 Troubleshooting . . . 25
12. Testing, limits and FAQ 26
12.1 Test suite . . . 26
12.2 Known limits . . . 26
12.3 FAQ . . . 26

1. Overview

ClickSense is an MCP server (Python, FastMCP) that lets an AI agent answer business questions on a ClickHouse warehouse. It addresses the three ways such agents usually fail:

- **They answer the wrong question.** Ambiguous words ("sales": revenue, orders or units?), values in the user's wording ("France" when the column stores `'FR'`) and re-invented metric formulas give plausible but wrong numbers. ClickSense ships a semantic layer and disambiguation tools that surface these issues before a query runs.
- **They write slow or broken SQL.** Unknown columns, missing partition filters, joins in the wrong order. ClickSense hands the agent the sorting keys, partitions and verified examples up front, lints SQL before execution and refuses queries whose index estimate exceeds a budget.
- **They can hurt the database.** A runaway loop of heavy queries can saturate a production cluster. ClickSense combines a constrained read-only account, a static guard, admission control, a circuit breaker and per-query limits.

1.1 Architecture



Every tool call goes through the MCP surface into `service.py`, which uses the semantic layer (files on disk) and the live catalog (ClickHouse system tables). Anything that reaches ClickHouse passes the guard and then the executor, which is the only code path that talks to the database.

1.2 Key capabilities

Capability	What it gives you
One-call context	`get_context` returns tables, columns, keys, metrics, glossary entries, values spotted in the question, ambiguities, verified examples, rules and suggested skills.
Governed metrics	`query_metric` compiles official formulas with resolved filter values, explicit periods and declared joins: no SQL written by the model.
Value resolution	User wording mapped to stored values (exact, case/accent/punctuation, synonyms, labels, fuzzy, live search) with an explicit status.
Pre-execution checks	Unknown columns with suggestions, literals absent from the data, performance findings, table caveats, index-based row estimate.

Capability	What it gives you
Human in the loop	`ask_user_to_clarify` uses MCP elicitation (both the legacy and the 2026-07-28 input-required flow) and degrades to a chat question.
Protection	Read-only account with constraints, guard, pre-flight, rate limit, concurrency slots, circuit breaker, per-query limits, query ids and audit log.
Speed	Connection pool + lz4, result cache, ClickHouse query cache, cached schema and value sets; typical calls take 2-40 ms.

2. Installation and quick start

2.1 Requirements

- Python 3.11+ and `uv` (or pip).
- A ClickHouse server reachable over HTTP (8123 / 8443). For the demo: the ClickHouse binary (`brew install --cask clickhouse`) or Docker.
- An MCP client: Claude Code, Claude Desktop or any client supporting stdio or streamable HTTP.

2.2 Install the server

```
cd clicksense
uv venv .venv
uv pip install --python .venv/bin/python -e ".[dev]"
```

2.3 Start the demo database

The repository contains a deterministic e-commerce dataset (customers, products, 3M order lines, 20M web events) and a read-only account configured like a production one.

```
scripts/dev-clickhouse.sh start     # ClickHouse on 127.0.0.1:18123 (HTTP) / 19000 (native)
scripts/dev-clickhouse.sh seed      # loads dev/seed.sql (~6 s)
# other commands: stop | status | reset
```

With Docker instead:

```
docker compose up -d
docker compose exec -T clickhouse clickhouse-client --multiquery < dev/seed.sql
```

2.4 Configure and check

```
cp clicksense.example.yaml clicksense.yaml
.venv/bin/clicksense --check          # connection, readonly level, limits, semantic layer counts
.venv/bin/python -m pytest -q         # 100 tests; live tests are skipped if ClickHouse is down
```

> **Tip**
> `--check` prints `"readonly": 2` for a correctly configured account. A value of 1 means per-query limits cannot be sent (see Chapter 9).

2.5 Connect an MCP client

Claude Code / Claude Desktop (stdio)

```
{
  "mcpServers": {
    "clicksense": {
      "command": "/path/to/clicksense/.venv/bin/clicksense",
      "args": ["--config", "/path/to/clicksense/clicksense.yaml"]
    }
  }
}
```

Streamable HTTP

```
.venv/bin/clicksense --config clicksense.yaml --transport http --host 127.0.0.1 --port 3110
# MCP endpoint: http://127.0.0.1:3110/mcp
```

> **Security**
> The HTTP transport has no authentication. Bind it to localhost or put it behind an authenticating reverse proxy.

2.6 Command-line reference

Option	Default	Meaning
--config PATH	$CLICKSENSE_CONFIG or ./clicksense.yaml	Configuration file.
--transport	stdio	stdio or http.
--host / --port	127.0.0.1 / 3110	HTTP bind address.
--check	off	Print the status report and exit.

3. Configuration

Configuration is one YAML file. `${VAR}` and `${VAR:-default}` are substituted from the environment; a variable without a default that is not set stops the server at start (no silent empty password). Relative paths are resolved against the directory of the configuration file.

```
connection:
  host: ${CLICKSENSE_HOST:-localhost}
  port: ${CLICKSENSE_PORT:-18123}
  username: ${CLICKSENSE_USER:-clicksense}
  password: ${CLICKSENSE_PASSWORD:-clicksense}
  database: shop
limits:
  default_rows: 200
  max_execution_time: 20
  preflight_max_rows: 300000000
policy:
  allowed_databases: [shop]
  pii: block
semantic_dir: semantic
audit_path: audit/clicksense.jsonl
```

3.1 connection

Key	Default	Meaning
host / port	localhost / 8123	ClickHouse HTTP interface.
username / password	default / ""	Use the dedicated read-only account.
secure	false	HTTPS (port 8443 usually).
database	default	Database for unqualified table names. Must not be a system database.
connect_timeout / send_receive_timeout	5 / 90 s	Socket timeouts.
pool_size	8	HTTP connection pool size.
compress	lz4	Transport compression.

Environment overrides: CLICKSENSE_HOST, CLICKSENSE_PORT, CLICKSENSE_USER, CLICKSENSE_PASSWORD, CLICKSENSE_DATABASE, CLICKSENSE_SECURE.

3.2 limits

Key	Default	Meaning
default_rows / hard_max_rows	200 / 5000	Rows returned by default / maximum a caller may request.
max_response_bytes	200000	Result payload budget; beyond it the result is truncated and says so.
max_sql_chars	20000	Longest statement accepted.
max_execution_time	20 s	Per-query time limit sent to ClickHouse.
max_memory_usage	1 GB	Per-query memory limit.
max_threads	4	Per-query threads.
max_rows_to_read / max_bytes_to_read	1e9 / 1e11	Read limits enforced by ClickHouse.
max_bytes_before_external_group_by / _sort	500 MB	Spill to disk instead of failing on memory.
preflight / preflight_max_rows	true / 3e8	EXPLAIN ESTIMATE before running; refuse above the budget.

Key	Default	Meaning
max_concurrent / queue_timeout_s	4 / 15 s	Client-side concurrency slots and queue wait.
rate_per_minute	240	Token bucket for queries per minute.
breaker_failures / breaker_cooldown_s	4 / 30 s	Circuit breaker on consecutive load failures.
result_cache_ttl_s / result_cache_entries	120 s / 256	In-process result cache.
use_query_cache / query_cache_ttl_s	true / 300 s	ClickHouse query cache.

Per-query limits can only *tighten* what the account profile allows: ClickHouse rejects any setting above a profile CONSTRAINT.

3.3 policy

Key	Default	Meaning
allowed_databases	[] (all non-system)	Databases the guard and catalog accept.
denied_tables	[]	"db.table" entries hidden and refused.
allowed_table_functions	[numbers]	Table functions permitted in FROM.
pii	block	block: PII columns may be filtered or aggregated but never returned raw. allow: no restriction.
server_parse_fallback	true	If sqlglot cannot parse valid ClickHouse SQL, ask ClickHouse (EXPLAIN AST).

3.4 Other keys

Key	Default	Meaning
semantic_dir	semantic	Location of the semantic layer.
audit_path	audit/clicksense.jsonl	JSONL audit log; null disables it.
catalog_ttl_s	600	Live schema cache.
value_cache_ttl_s	3600	Cache of column value enumerations.
value_cache_max_distinct	2000	Above this, a column is high-cardinality and searched live.
value_scan_max_rows	5e7	Larger tables are sampled or time-windowed when enumerating values.

4. The recommended agent workflow

The server instructions (sent to every client) and the `answer-data-question` skill describe the same loop. Following it is what makes answers reliable.

Step	Tool(s)	Why
1. Context	get_context	One call returns everything relevant, including ambiguities and examples.
2. Ambiguity	ask_user_to_clarify	Never pick a meaning silently. Use the context first; ask when it does not settle the question.
3. Period	check_freshness	State the period used. Check freshness before "today" / "yesterday".
4a. Metric	query_metric	When a governed metric fits: official formula, resolved values.
4b. SQL	describe_table, resolve_values, validate_query, run_query	Exact names, real literals, checked before running.
5. Check	(result fields)	truncated = incomplete; 0 rows = read notes first; sanity-check magnitudes.
6. Answer	suggest_knowledge	Give numbers with the definition, filters, period and caveats; propose reusable learnings.

4.1 Worked example

Question: *"What were sales in France on the website last month?"*

get_context ranks `shop.orders` first, proposes the metrics revenue, orders and units_sold, spots `website` → `channel = 'web'` and `France` → `country = 'FR'`, and reports an ambiguity: **sales** has three senses. The agent asks; the user picks revenue. Then:

```
query_metric {"metrics": ["revenue"], "dimensions": ["channel"],
              "filters": {"country": "France"}, "period": "last_month"}
```

Real response (abridged):

```
{ "ok": true, "columns": ["channel", "revenue"], "row_count": 4,
  "rows": [["web", 13949044.75], ["app", 9209137.36], ["store", 4835205.24],
           ["marketplace", 4666537.69]],
  "truncated": false,
  "metric_definitions": [{"name": "revenue", "expression": "sum(o.amount_eur)", "unit": "EUR"}],
  "period": {"from": "2026-09-01", "to_exclusive": "2026-10-01", "column": "order_date"},
  "value_resolutions": [{"dimension": "country", "from": "France", "to": "FR", "match": "label"}],
  "sql": "SELECT o.`channel` AS `channel`, sum(o.amount_eur) AS `revenue`
          FROM `shop`.`orders` AS o
          WHERE (o.status = 'completed')
            AND o.`order_date` >= {p_from:Date} AND o.`order_date` < {p_to:Date}
            AND o.`country` IN {f0:Array(String)}
          GROUP BY `channel` ORDER BY `revenue` DESC LIMIT 100",
  "params": {"p_from": "2026-09-01", "p_to": "2026-10-01", "f0": ["FR"]} }
```

Had the agent written `WHERE country = 'France'` by hand, `run_query` would have returned zero rows with this note:

```
{"severity": "error", "code": "unknown_value",
 "message": "'France' does not occur in shop.orders.country.", "fix": "Use 'FR'.",
 "candidates": [{"value": "FR", "score": 0.96, "match": "label", "label": "France", "rows": 1064411}]}
```

5. Tool reference

All tools return JSON objects with `"ok": true` on success. A refusal or failure returns `isError: true` with `{ok: false, error, message, hint, ...}`: `error` is a stable code (see section 11.2), `hint` says what to do instead. Every tool declares `readOnlyHint` (true except `suggest_knowledge` and `cancel_query`) and `openWorldHint: false`.

5.1 Discovery

`get_context`

Start here. Everything relevant to a question in one call.

Argument	Type / default	Meaning
question	string	The user's question, verbatim.
limit	int = 6 (1-20)	Caps tables (max 3), metrics and examples.

Returns. `tables` (summary, time column, sorting key, brief columns, joins, caveats), `metrics`, `glossary`, `values_spotted`, `ambiguities` (business terms with several senses, values present in several columns), `examples` (verified SQL), `knowledge` excerpts, `skills`, `next_steps`.

Notes. Tables are ranked from lexical matches on the live schema and dictionary, boosted by matching metrics and glossary terms and by values found in the question. Two-letter words count as values only when written in capitals ("CA", not "de").

`list_tables`

Tables available, with description, row count and grain.

Argument	Type / default	Meaning
search	string?	Keywords to filter.

Returns. `tables`; `warning` when the live catalog is unavailable; `dictionary_only` lists tables documented but absent from ClickHouse.

`describe_table`

Full description of one table.

Argument	Type / default	Meaning
table	string	database.table or a table of the default database.
include_samples	bool = false	Adds 5 sample rows (PII hidden).

Returns. Engine, size, sorting/partition/sampling keys, time column, every column (type, description, keys, unit, PII, values, value synonyms, caveats), skip indices, joins, dimensions, related knowledge and metrics, `performance_hints`.

Notes. Unknown names return candidates ("Did you mean ...").

`search_knowledge`

BM25 search over the whole semantic layer.

Argument	Type / default	Meaning
query	string	Free text.
kinds	list?	table, column, metric, term, knowledge, skill, example.
limit	int = 8	Maximum results.

Returns. `results` with kind, id, title, score and a kind-specific payload (excerpt, definition and senses, expression, SQL...).

`get_knowledge`

Full text of one knowledge document.

Argument	Type / default	Meaning
doc_id	string	From get_context or search.

Returns. id, title, tags, applies_to, body.

`list_skills / get_skill`

List the playbooks; return one playbook's step-by-step instructions.

Argument	Type / default	Meaning
name	string	get_skill only.

Returns. name, description, when_to_use, body.

`list_metrics`

Governed metrics.

Returns. name, label, description, table, unit, dimensions, synonyms.

5.2 Querying

`query_metric`

Compute governed metrics without writing SQL.

Argument	Type / default	Meaning
metrics	list[string]	Names or synonyms, e.g. ["revenue", "orders"]. Same table and built-in filters.
dimensions	list?	Group-by dimensions: table columns or declared dimensions (joins added automatically).
filters	object?	dimension → value | [values] | {op: value}, op in = != > >= < <= in not_in. User wording is resolved to stored values.
period	string | object?	today, yesterday, this_week, last_week, this_month/mtd, last_month, this_quarter/qtd, last_quarter, this_year/ytd, last_year, last_N_days|weeks|months, or {from, to} (inclusive).
grain	enum?	hour, day, week (Monday), month, quarter, year.
order_by / descending	string? / true	A metric, a dimension or 'period'.
limit	int = 100	Rows.
format	json | markdown | csv	Output format.

Returns. Same fields as run_query plus `metric_definitions`, `period`, `value_resolutions`, `sql` and `params`.

Notes. An ambiguous or unknown filter value stops compilation with `needs_clarification` and the candidates: the compiler never guesses. Values are always bound as typed parameters.

`run_query`

Run one read-only SELECT through the guard, the pre-flight and the executor.

Argument	Type / default	Meaning
sql	string	SELECT, WITH ... SELECT or UNION. One statement.
params	object?	Values for {name:Type} placeholders.
max_rows	int? (1-5000)	Default 200.
format	json | markdown | csv	Default json.

Returns. `columns`, `types`, `rows` (or table / csv), `row_count`, `truncated` (+ reason and hint), `row_cap`, `stats` (query_id, elapsed_ms, read_rows, read_bytes, cached), `notes` (errors/warnings from the advisor), `empty_hint`.

```
run_query {"sql": "SELECT device, uniq(session_id) AS sessions FROM shop.web_events
                   WHERE event_date >= {d:Date} GROUP BY device ORDER BY sessions DESC",
           "params": {"d": "2026-10-01"}}
```

`sample_rows`

A few raw rows to see the data.

Argument	Type / default	Meaning
table	string	
n	int = 10 (1-100)	
columns	list?	
where	string?	SQL condition without WHERE (guarded).

Returns. As run_query, plus `hidden_pii_columns`.

5.3 Control

`validate_query`

Check a query without running it.

Argument	Type / default	Meaning
sql	string	
params	object?	
check_values	bool = true	Look up filter literals.

Returns. `valid`, `verdict` (ok | review_warnings | fix_before_running | rejected), `findings`, `estimate`, `estimated_rows`, `sql_to_run`, `parser`.

Findings raised by the advisor:

Code	Severity	Meaning
unknown_column	error	Column absent; suggestions include case-only differences and synonyms.
ambiguous_column	warning	Unqualified column present in several joined tables.
unknown_value	error / warning	Filter literal not stored in the column; candidates given.
no_partition_filter	warning	Large partitioned table without a filter on its time column.
primary_key_unused	info	No filter on the first ORDER BY key.
select_star	warning	SELECT * on a column store.
join_order	warning	Right side of a JOIN larger than the left (hash table built from the right).
final / count_distinct / leading_wildcard	info	Costly constructs, with the cheaper alternative.
too_expensive	error	Estimated rows to read above the pre-flight budget.
caveat	info	Caveats documented for the tables used.

`explain_query`

Execution plan with index usage, the estimate, and optimisation findings.

Argument	Type / default	Meaning
sql	string	
params	object?	

Returns. `plan` (EXPLAIN indexes = 1, truncated at 8000 chars), `estimate`, `estimated_rows`, `findings`.

`profile_column`

Bounded statistics for one column.

Argument	Type / default	Meaning
table	string	
column	string	

Returns. rows, nulls, distinct_approx, min/max, avg and p50/p90/p99 (numeric), empty and avg_length (text), top_values (not for PII); `scope` says full, sample or last_90_days.

`check_freshness`

Latest timestamp and lag.

Argument	Type / default	Meaning
table	string	

Returns. latest, server_now, lag_hours, expected_refresh (from the dictionary).

5.4 Disambiguation

`resolve_values`

Map user wording to stored values of one column.

Argument	Type / default	Meaning
table	string	
column	string	
terms	list[string]	As the user said them.

Returns. Per term: `status` (exact | resolved | ambiguous | not_found), `value` to use when resolved, `candidates` with score, match type, label and row count, `coverage`; `all_resolved` overall.

```
{"term": "canceled", "status": "resolved", "value": "cancelled",
 "candidates": [{"value": "cancelled", "score": 0.96, "match": "synonym",
                 "label": "Cancelled before shipping", "rows": 150027}], "coverage": "full"}
```

`locate_value`

Which table/column stores a value (low-cardinality columns).

Argument	Type / default	Meaning
term	string	e.g. 'Lyon'.
tables	list?	Restrict the search.

Returns. `matches` (table, column, value, score, rows); `ambiguous` when the value exists in several columns.

`find_columns`

Which columns a business word may refer to.

Argument	Type / default	Meaning
term	string	

Returns. Ranked `candidates`; `ambiguous` when several fit closely.

`ask_user_to_clarify`

Ask the user to choose between interpretations.

Argument	Type / default	Meaning
question	string	Short question.
options	list[string] (2-8)	Candidate meanings or values.

Returns. `answered` + `choice`; `declined` / `cancel`; or `needs_clarification` when the client cannot elicit, in which case the agent asks in chat.

Notes. Legacy clients get a server-initiated elicitation. On the 2026-07-28 protocol the tool returns an InputRequiredResult and reads the answer when the client calls back. Clients that do not advertise elicitation are never sent a request.

5.5 Operations and learning

`server_status`

Health and protection state.

Returns. ClickHouse reachability, version, readonly level; breaker state, failures, slots, running queries; per-query limits; cache statistics; totals (queries, errors, rows and bytes read); catalog and knowledge counts and load errors.

`cancel_query`

Cancel a running query started by this server.

Argument	Type / default	Meaning
query_id	string	From stats or status.

Returns. `kill requested`. Ids not issued by this server are refused.

`refresh_catalog`

Reload semantic files and the live schema; clear caches.

Returns. Table count and load errors.

`suggest_knowledge`

Propose an addition for human review.

Argument	Type / default	Meaning
kind	example | term | caveat | metric | note	
title	string	
content	string	
sql	string?	For examples.

Returns. The inbox file written. Never used until a person moves it into the semantic layer.

6. Resources, prompts and skills

6.1 Resources

URI	Content
clicksense://guide	Server instructions and SQL tips (markdown).
clicksense://catalog	All tables (JSON).
clicksense://tables/{database}/{table}	One table, as describe_table (JSON).
clicksense://glossary	The glossary (JSON).
clicksense://metrics	Metric definitions (JSON).
clicksense://knowledge/{doc_id}	One knowledge document (markdown).
skill://{name}/SKILL.md	Native MCP skills (FastMCP skills provider), plus a _manifest per skill.

6.2 Prompts

- `analyze(question)` — runs the recommended workflow on a question.
- `skill-<name>(question)` — one prompt per skill, with the playbook body.

6.3 Skills shipped

Skill	Use it for
answer-data-question	Any question needing a number, a list or a trend.
investigate-metric-change	Why a metric rose or dropped: confirm, rule out freshness, decompose by dimension, split volume vs value, drill down, report drivers.
data-quality-check	Freshness, nulls, duplicates on the grain, referential integrity.
write-fast-clickhouse-sql	Partition/primary-key filters, approximate functions, conditional aggregates, join order, SAMPLE.
disambiguate-request	Turn a vague request into precise parameters before querying.

Skills are exposed three ways — as tools (list_skills / get_skill), as prompts and as native MCP skills — so every client can use them. They follow the Agent Skills format and can also be copied into `~/.claude/skills`.

7. Authoring the semantic layer

Everything lives in `semantic/` as plain files: review them in pull requests like code. Files are re-read automatically when they change (checked at most every 2 seconds); `refresh_catalog` forces a reload. Load errors are reported by `server_status` instead of crashing the server.

```
semantic/
  dictionary/*.yaml      tables, columns, value labels and synonyms, joins, dimensions, caveats
  glossary.yaml          business terms; several senses = ambiguous
  metrics.yaml           governed metrics
  examples.yaml          verified question -> SQL pairs
  knowledge/*.md         rules, conventions, calendars (YAML front-matter)
  skills/<name>/SKILL.md playbooks (Agent Skills format)
  inbox/                 suggestions from the agent, for review
```

Where to start
Without any file the server already works from ClickHouse COMMENTs and the live schema. The highest-value additions, in order: **time_column** and **caveats** of the main tables, **values / value_synonyms** of frequently filtered columns, then **3-5 metrics** and a few verified examples.

7.1 Data dictionary

```
tables:
  shop.orders:
    description: Order lines (one row per product per order), all channels.
    grain: one row per ordered product
    alias: o                       # used by query_metric
    time_column: order_date        # default period column
    refresh: daily, around 06:00 Europe/Paris
    caveats:
      - Official revenue only counts status = 'completed'.
    joins:
      customers: {table: shop.customers, alias: c, on: "o.customer_id = c.customer_id"}
    dimensions:
      segment: {sql: c.segment, join: customers, column: shop.customers.segment}
    columns:
      status:
        description: Order status
        values: {completed: Delivered and paid, cancelled: Cancelled before shipping}
        value_synonyms:
          cancelled: [canceled, cancellation, annulée]
      email: {description: Email address, pii: true}
```

Table key	Meaning
description, grain, owner, tags, synonyms	Shown to the agent and indexed for search.
alias	Table alias in compiled metric SQL (default t).
time_column	Period filters, grains, freshness, partition lint. Inferred from the partition key if absent.
refresh	Expected cadence, returned by check_freshness.
caveats	Returned by get_context, describe_table and as validate_query findings.
joins	name → {table, alias, on, type (default LEFT)}. Used by dimensions.
dimensions	name → {sql, join?, column?, description}. column links to the stored values for resolution.

Column key	Meaning
description, unit, synonyms, caveats	Documentation and search.
pii: true	Never returned raw under policy pii: block; not enumerated by the resolver.
values	stored value → human label. Labels are matched by the resolver.

Column key	Meaning
value_synonyms	stored value → list of user words (may be multilingual).
values_complete	Default true: values is the complete list (used to lint literals without a query).
enumerable	Force value enumeration for a non-LowCardinality column.

YAML pitfall handled
In YAML 1.1, on, yes, no are booleans. ClickSense's loader only treats true/false as booleans, so a join's `on:` key works as written.

7.2 Glossary

```
terms:
  - term: sales
    definition: Ambiguous word — it can mean money, orders or units.
    senses:
      - {meaning: Revenue in EUR, maps_to: "metric:revenue"}
      - {meaning: Number of orders, maps_to: "metric:orders"}
  - term: revenue
    synonyms: [turnover, GMV]
    definition: Sum of amounts incl. VAT of delivered order lines.
    maps_to: "metric:revenue"
```

A term with several `senses` is reported in `ambiguities` whenever it appears in a question. `maps_to: "metric:x"` boosts that metric and its table in get_context.

7.3 Metrics

```
metrics:
  - name: revenue
    label: Revenue
    description: Sum of amounts including VAT of delivered order lines.
    table: shop.orders
    expression: sum(o.amount_eur)          # an aggregate over the table alias
    filters: ["o.status = 'completed'"]    # always applied
    time_column: order_date
    unit: EUR
    synonyms: [turnover, gmv]
    dimensions: [country, channel, category, segment]   # allowed (table columns always allowed)
```

- Metrics combined in one call must share the table and the built-in filters.
- Ratios are fine: `round(sum(o.amount_eur) / count(), 2)`.
- Expressions and filters are trusted (authored by people, reviewed in git); filter *values* from the agent are always bound as parameters.

7.4 Examples and knowledge

```
examples:
  - question: Top 10 products by revenue last month
    tags: [top, products, revenue]
    sql: |
      SELECT p.name, sum(o.amount_eur) AS revenue
      FROM shop.orders AS o INNER JOIN shop.products AS p ON o.product_id = p.product_id
      WHERE o.status = 'completed' AND o.order_date >= toStartOfMonth(today() - toIntervalMonth(1))
        AND o.order_date < toStartOfMonth(today())
      GROUP BY p.name ORDER BY revenue DESC LIMIT 10
    notes: The small table is on the right of the JOIN.
```

```
---
id: revenue-rules
title: Revenue calculation rules
tags: [revenue, finance, vat]
applies_to: [shop.orders]        # listed by describe_table for these tables
---
Official revenue is the sum of amount_eur over lines with status = 'completed'. ...
```

7.5 Skills

```
semantic/skills/investigate-metric-change/SKILL.md
---
name: investigate-metric-change
description: Find what drives a rise or drop of a metric between two periods.
when_to_use: Why did revenue drop or rise, variance analysis, anomaly.
---
1. Pin the metric and the two periods ...
```

7.6 The inbox loop

When the agent discovers something reusable (a verified query, a definition, a caveat) it calls `suggest_knowledge`. A markdown file lands in `semantic/inbox/` with `status: proposed`. A person reviews it and moves the content into the right file. Nothing in the inbox is ever used automatically, so a wrong conclusion cannot poison future answers.

8. Disambiguation in depth

Most wrong answers from data agents are not SQL errors: the query runs and returns a plausible number for a slightly different question. ClickSense treats ambiguity as a first-class result.

8.1 Kinds of ambiguity and how each is surfaced

Ambiguity	Example	Surfaced by
Business term with several meanings	"sales", "CA"	glossary senses → get_context ambiguities
Column choice	"country": delivery vs customer	glossary, find_columns, values in several columns
Value wording	"France" vs 'FR', "canceled" vs 'cancelled'	resolve_values, query_metric, unknown_value lint
Value shared by columns	a city that is also a store name	locate_value, get_context
Period	"recently", "this year"	explicit period in query_metric responses; skill guidance
Data not yet loaded	"today" on a daily batch	check_freshness, knowledge 'data-calendar'

8.2 How values are resolved

Sources, cheapest first:

- **Dictionary** labels and value synonyms — no query.
- **Cached enumeration** of the column's distinct values with frequencies (up to 2000), computed with a bounded query — sampled (SAMPLE) or windowed (last 365 days) on very large tables — and cached for an hour.
- **Live search** for high-cardinality text columns: accent/case-insensitive contains, then n-gram distance; 20 most frequent matches, bounded by read limits.

Match	Score	Example
exact	1.00	'FR' → 'FR'
case/accent/punctuation	0.97	'saint etienne' → 'Saint-Étienne'
label / synonym	0.96	'France' → 'FR', 'business' → 'B2B'
prefix / contains	0.82 / 0.75	'Saint-É' → 'Saint-Étienne'
fuzzy (WRatio ≥ 72)	≤ 0.88	'Lyonn' → 'Lyon'
live_search	0.65-0.97	names in a 10M-row column

Status	Rule	What the agent should do
exact	Top score 1.0	Use the value.
resolved	One candidate ≥ 0.9 clearly ahead	Use value; mention the mapping in the answer.
ambiguous	Several strong candidates, or best < 0.9	Ask the user with the candidates.
not_found	No candidate	If coverage is complete, the value does not occur: say so. Else try another spelling or locate_value.

9. Safety model: not taking the database down

> **Principle**
> The ClickHouse account is the security boundary. Everything in the MCP server is defence in depth: it prevents mistakes and load spikes, and explains refusals so the agent can fix its query, but it is not a substitute for a read-only account with constraints.

9.1 Line 1 — the ClickHouse account

`dev/production-account.sql` creates the production setup with SQL-driven access control (`dev/clickhouse/users.xml` is the XML equivalent used by the demo):

```
CREATE SETTINGS PROFILE IF NOT EXISTS clicksense_ro SETTINGS
    readonly = 2 CONST,  allow_ddl = 0 CONST,  allow_introspection_functions = 0 CONST,
    max_execution_time = 30 MAX 60,          max_memory_usage = 2000000000 MAX 4000000000,
    max_threads = 4 MAX 8,                   max_rows_to_read = 2000000000 MAX 5000000000,
    max_concurrent_queries_for_user = 8 MAX 8,  priority = 5 MIN 5,
    max_bytes_before_external_group_by = 1000000000,
    cancel_http_readonly_queries_on_client_close = 1;
CREATE QUOTA clicksense_quota FOR INTERVAL 1 hour
    MAX queries = 5000, errors = 1000, read_rows = 200000000000, execution_time = 7200 TO clicksense;
CREATE USER clicksense IDENTIFIED WITH sha256_password BY 'change-me'
    HOST IP '10.0.0.0/8' SETTINGS PROFILE 'clicksense_ro';
GRANT SELECT ON shop.* TO clicksense;
GRANT SELECT ON system.data_skipping_indices TO clicksense;
```

- **readonly = 2**: data can be read and settings changed — required so the server can send tighter per-query limits. readonly = 1 would silently drop them.
- **MAX / MIN / CONST** constraints: no client (and no model) can raise time, memory, threads or priority.
- **priority = 5 MIN 5**: agent queries always yield to interactive and ETL workloads.
- **Quota**: an hourly ceiling on queries, errors, rows read and execution time.
- Optional on recent versions: a dedicated workload (CREATE WORKLOAD) with its own CPU share.
- Verified by the test suite: with the guard bypassed, CREATE, INSERT, DROP and SETTINGS max_threads = 64 are refused by ClickHouse itself.

9.2 Line 2 — the server, before sending

Static guard

Rule	Refusal code
Exactly one statement (a ';' outside string literals is refused)	multiple_statements
Only SELECT / WITH ... SELECT / UNION / INTERSECT / EXCEPT	not_a_select
No SETTINGS, FORMAT or INTO OUTFILE clause	settings_clause, format_clause, into_clause
No external or time-burning functions: url, s3, gcs, hdfs, file, remote, cluster, mysql, postgresql, jdbc, odbc, mongodb, redis, sqlite, executable, input, iceberg, deltaLake, hudi, merge, generateRandom, sleep, sleepEachRow, dictionary, ...	forbidden_function
No system / information_schema tables (use the tools)	system_table
Database allow-list and table deny-list	database_not_allowed, table_denied
PII columns not selected raw (SELECT * included; aggregates allowed)	pii_column
Length limit	too_long

Parsing uses sqlglot. For valid ClickHouse syntax that sqlglot does not know (for example `LIMIT 1 BY x LIMIT 5`), the guard asks ClickHouse's own parser (`EXPLAIN AST`, parse only) and applies the same table and function rules to its tree.

Row cap by wrapping

A query without a LIMIT, or with one above the cap, runs as `SELECT * FROM (<your sql>) LIMIT cap+1`. The model's text is never regenerated, and fetching cap+1 rows tells the server reliably whether the result was truncated.

Pre-flight estimate

When the tables referenced are large enough to exceed the budget even in theory, the server runs `EXPLAIN ESTIMATE` (primary index analysis, no data read). Above `preflight_max_rows` the query is refused with `too_expensive`, the estimate per table and the advisor's findings (typically: add a filter on the partition column). Small tables skip the round trip.

Admission control and circuit breaker

- **Token bucket**: rate_per_minute queries; above it, rate_limited with a retry delay.
- **Concurrency slots**: max_concurrent queries in flight; others wait up to queue_timeout_s, then busy.
- **Circuit breaker**: after breaker_failures consecutive *load* failures (timeout, memory limit, too many simultaneous queries, quota, network) the server stops sending queries for breaker_cooldown_s and answers circuit_open. SQL errors and bounded refusals (too many rows to read) do not count: bad SQL is not a reason to stop talking to the database.

9.3 Line 3 — every query carries its limits

Setting sent	Purpose
max_execution_time + timeout_overflow_mode=throw	Hard time limit.
max_memory_usage, max_threads	Resource caps per query.
max_rows_to_read, max_bytes_to_read (read_overflow_mode=throw)	Read caps enforced while reading.
max_bytes_before_external_group_by / _sort	Spill to disk instead of OOM.
max_result_rows	Backstop to the row cap.
cancel_http_readonly_queries_on_client_close	A dropped client cancels its query.
query_id = cs-<uuid>	Cancellation by id (cancel_query) and tracing.
log_comment = clicksense:<tool>	Find agent queries in system.query_log.
use_query_cache (+ nondeterministic/system handling = ignore)	ClickHouse-side result reuse.
wait_end_of_query = 1	Errors arrive as HTTP errors; exact read statistics.

9.4 PII

Columns marked `pii: true` can be filtered and aggregated (count, uniq) but not returned raw; `sample_rows` drops them; the resolver does not enumerate them; the audit log never contains parameter values.

10. Performance and tuning

10.1 Measured latencies

Demo database on a MacBook, local ClickHouse 26.10, through the stdio transport:

Call	Latency
get_context, cold (enumerates LowCardinality values)	≈ 280 ms
get_context, warm	≈ 2 ms
query_metric, revenue by channel, 1 month (87k rows read)	≈ 35 ms
same call from the result cache	≈ 2.5 ms
run_query, 30 days of web_events (585k rows read)	≈ 13 ms
full scan of 20M rows	≈ 110 ms

10.2 What makes it fast

- Pooled HTTP connections with lz4 compression and no server sessions (sessions serialise queries).
- In-process result cache (TTL + LRU) and the ClickHouse query cache.
- Catalog, value sets and search indexes kept in memory; the semantic layer is parsed once per change.
- Server-side parameter binding ({name:Type}) — no string formatting, plan-friendly.
- Compiled metrics filter on the time column, which is the partition key and the ORDER BY prefix.
- Above all: better queries. The agent receives sorting keys, partitions, verified examples and lint findings before it runs anything.

10.3 Tuning guide

Symptom	Adjust
Legitimate queries refused as too_expensive	Raise limits.preflight_max_rows, or document a time_column so metrics and lint push partition filters.
Timeouts on heavy but valid analyses	Raise max_execution_time (within the profile MAX) or pre-aggregate with a materialized view and document it.
busy / rate_limited during agent bursts	Raise max_concurrent / rate_per_minute — only if the cluster has headroom.
Slow first get_context on a wide warehouse	Restrict allowed_databases; mark only useful columns enumerable; the cost is paid once per value_cache_ttl_s.
Stale answers	Lower result_cache_ttl_s / query_cache_ttl_s, or set use_query_cache: false.

11. Operations and troubleshooting

11.1 Audit log

One JSON line per tool call that touched ClickHouse or was refused: timestamp, tool, status, SQL (truncated), parameter *names*, query_id, rows, rows/bytes read, cache hit, duration, error code. Join it with `system.query_log` on query_id for server-side details.

```
{"ts": "2026-10-31T23:07:22+0100", "tool": "run_query", "status": "ok", "sql": "SELECT ...",
 "params": ["d"], "query_id": "cs-1bdaa266-...", "rows": 3, "read_rows": 585192,
 "read_bytes": 6437163, "cached": false, "ms": 13.1}
```

```
-- all agent queries in the last hour, slowest first
SELECT query_id, log_comment, query_duration_ms, read_rows, memory_usage, exception
FROM system.query_log
WHERE log_comment LIKE 'clicksense:%' AND event_time > now() - INTERVAL 1 HOUR AND type != 'QueryStart'
ORDER BY query_duration_ms DESC LIMIT 20
```

11.2 Error codes

Code	Origin	Meaning and fix
timeout, memory_limit	ClickHouse 159 / 241	Filter on partition/primary key, aggregate earlier, smaller JOIN right side. Counts for the breaker.
too_many_rows_to_read / _bytes_	ClickHouse 158 / 307	Read limit hit. Add filters or SAMPLE.
server_busy, quota_exceeded, network	ClickHouse 202 / 201 / 210	Load or availability. Counts for the breaker.
unknown_identifier, unknown_table, unknown_function	ClickHouse 47 / 60 / 46	Use describe_table; identifiers and functions are case-sensitive. Findings are attached when possible.
syntax_error, type_error, type_mismatch, not_an_aggregate	ClickHouse 62 / 43 / 53 / 215	SQL to fix.
access_denied, readonly, setting_constraint	ClickHouse 497 / 164 / 452	The account refused: by design.
too_expensive	pre-flight	Estimate above budget; see findings.
circuit_open, busy, rate_limited	executor	Back off and retry later with a cheaper query.
needs_clarification	query_metric	Ambiguous value: ask the user with the candidates.
unknown_metric / _dimension, mixed_tables, mixed_filters, bad_period, bad_grain, bad_filter	metrics	The message lists valid options.
guard codes	guard	See section 9.2.

11.3 Troubleshooting

Problem	Check
--check shows reachable: false	Host/port (HTTP port, not 9000), credentials, network ACL (HOST IP of the user).
readonly: 1 warning	Set readonly = 2 in the profile; with 1, per-query limits are ignored.
setting_constraint on every query	A limit in clicksense.yaml exceeds the profile MAX: lower it.
Live catalog unavailable	The user needs SELECT on the allowed databases; system.tables/columns are readable by default.
Knowledge errors in server_status	Invalid YAML or front-matter; the file is skipped and named.
Elicitation never shown	The client does not advertise elicitation: the tool returns needs_clarification and the agent must ask in chat.

12. Testing, limits and FAQ

12.1 Test suite

File	Covers
tests/test_guard.py	55 cases: allowed queries, attacks (writes, stacked statements, SETTINGS, external functions, system tables, PII), row cap, server-parse fallback.
tests/test_semantic.py	Loader, search, glossary, resolver statuses, metric compiler, periods — offline.
tests/test_mcp.py	readOnlyHint on every tool, isError on refusals, resources, prompts, native skills, elicitation with and without client support.
tests/test_live.py	Against the demo database: limits, account boundary, pre-flight, breaker, slots, cancellation, cache, end-to-end tools, audit content. Skipped when ClickHouse is down.

12.2 Known limits

- The guard is defence in depth, not a security boundary (section 9).
- EXPLAIN ESTIMATE covers MergeTree tables only; other engines are not pre-estimated (limits still apply).
- Column linting is skipped for queries over subqueries or CTEs; ClickHouse's error is translated instead.
- Search is lexical (BM25) and offline: a synonym missing from the dictionary will not be guessed.
- No authentication on the HTTP transport.
- Metrics combine only when they share a table and built-in filters.

12.3 FAQ

Can the agent write data? No. The guard only accepts SELECT, and the account is read-only. The only write is suggest_knowledge, which creates a file in semantic/inbox/.

Does it need an LLM or embeddings service? No. Everything runs locally and offline.

How do I add a second database? Grant SELECT on it to the account, add it to policy.allowed_databases and add a dictionary file for it.

Can synonyms be in several languages? Yes. Synonyms are user vocabulary; the demo keeps a few French words to show it.

Why wrap the query instead of adding LIMIT? So the SQL that runs is the SQL the model wrote, parameters and comments included, and truncation is detected exactly.